

VecScaleDB

A Purpose-Built Distributed Vector Database System

Team Name Deadlock and Chill	Reference Milvus, SIGMOD 2021	Team Members Sanchit Kumar Priyanshu Jha Rohini Mandlekar	Duration 5-6 weeks
--	---	---	------------------------------

1. Overview:

VecScaleDB is a distributed vector database built from scratch to reproduce the architectural scaling claims of Milvus (SIGMOD '21). While single-node libraries like Faiss handle indexing, modern AI applications—from YouTube's real-time recommendations to Airbnb's search rankings—require systems that manage billions of vectors across multiple nodes with low latency and consistent concurrent writes.

The project's core contribution is validating near-linear throughput scaling by implementing a shared-storage architecture that separates stateless computing from a durable LSM-based storage layer. By replicating the original paper's experiments at a 1-to-5-reader scale, VecScaleDB tries demonstrates that adding nodes increases QPS proportionally, proving that distributed mechanisms like consistent hashing, snapshot isolation, and scatter-gather execution can maintain both correctness and performance under load.

2. System Architecture:

VecScaleDB follows the shared-storage distributed design described in the Milvus paper. The cluster has three distinct roles, matching the paper's architecture exactly:

Role	Responsibility
Coordinator (×3, HA)	It keeps track of where every piece of data (shard) lives using a consistent hashing ring . It tells clients which Writer to send new data to and which Reader has the answer to their searchBy using etcd to manage three copies, the "brain" stays alive even if one instance crashes.
Writer (×1)	It is the only part of the system that handles new data, deletions, or updates.Every change is first saved to a "to-do list" (WAL) so no data is lost if power cuts out. It collects new vectors in memory and periodically saves them as permanent, immutable Segments to shared storage.
Reader (×1–5)	These nodes don't "own" data permanently; they just pull the Segments they need from shared storage into their own RAM when asked to search.Each Reader handles a specific slice of the total data. When a search comes in, they work together to find the best matches.

3. Features:

The evaluation is structured around reproducing and validating the distributed scalability result from Figure 10b of the Milvus paper. The primary artifact is a QPS-vs-nodes curve measured on a controlled workload. Secondary artifacts cover index correctness, fault tolerance, and system overhead.

Features	Description
Distributed & Scalable Architecture	Uses multiple reader nodes with shared storage and stateless compute design. Supports sharding and load balancing
Vector similarity search	Finds the most similar vectors using efficient indexes (IVF_FLAT). You can tune accuracy vs speed using a parameter (nprobe).

Dynamic data management	Supports inserting and deleting data in real-time using a pipeline that stores data in memory first and then writes it to disk efficiently.
Snapshot isolation	Ensures queries always see a consistent version of data, even when updates are happening at the same time
Fault tolerance	If a node crashes, it can recover quickly by reloading data from shared storage without data loss.
Distributed 5-node cluster	Uses multiple machines (1 writer + multiple readers) to process queries faster and scale performance

***Note:** Features may be added or adjusted during development based on hardware constraints, performance, and feasibility, with priority on stability and efficiency.*

4. Datasets:

All datasets are public standard ANN benchmarks used in the Milvus paper itself, enabling direct comparison of our results against the published figures.

Dataset	Vectors	Dims	Metric	Role
SIFT10K	10,000	128	L2	Unit tests, correctness (fast to reload)
SIFT1M	1,000,000	128	L2	Primary performance benchmark, Weeks 1–3
Deep1M	1,000,000	96	L2	Paper replication; same data as Milvus Fig. 8

5. Four-Phase Milestones:

The project is sequenced so that a working, correct single-node system exists before any distribution is attempted. Distributed complexity is introduced only once the correctness invariants are locked down by a comprehensive test suite.

Phase	Theme	Milestone
1	Foundation	Build core indexing using IVF_FLAT and a REST API for basic vector searches. Use the SIFT1M dataset to ensure high accuracy and recall.
2	Single-node durability	Implement a Write-Ahead Log (WAL) and LSM-based flushes to prevent data loss.Add snapshot isolation and segment merging for safe, concurrent reading and writing.
3	Distribution core	Split the system into a single Writer and multiple Reader nodes for scaling.Use consistent hashing and scatter-gather logic to coordinate searches across the cluster
4	Full evaluation	Scale to 5 Readers to prove that speed increases as you add nodes.Test fault tolerance by crashing and adding nodes while the system is running

6. Reference:

Jianguo Wang et al. "Milvus: A Purpose-Built Vector Data Management System." Proceedings of the 2021 ACM SIGMOD International Conference on Management of Data, June 2021, pp. 2614–2627.
<https://doi.org/10.1145/3448016.3457550>